

SYSTEM DESIGN & ARCHITECTURE

Sk6.0

Mobile & Backend Architecture Design

Project	Sk6.0 Lottery Platform
Objective	Scalable Web + Mobile system for 1,000+ concurrent users
Delivery Deadline	30 June 2026
Status	Advance received — Development in progress
Stack	FastAPI + PostgreSQL + Redis + Celery + Flutter
Document	System Design v1.0 — Confidential

Contents

1. Executive Summary & Design Goals
2. High-Level System Architecture
3. Backend — Clean Architecture
4. Backend — Folder Structure & Code Layout
5. Database Design & Transaction Strategy
6. API Design Standards
7. Authentication & Authorization
8. Mobile App — Clean Architecture (Flutter)
9. Mobile App — State Management & Offline
10. Scaling Strategy — Handling 1,000+ Users
11. Caching Strategy (Redis)
12. Background Jobs (Celery)
13. Security Architecture
14. Observability — Logs, Metrics, Alerts
15. Deployment Architecture
16. Development Workflow & CI/CD
17. Performance Targets & Load Testing
18. Risk Mitigation
19. Implementation Roadmap — 10 Week Sprint Plan

1. Executive Summary & Design Goals

This document describes the complete system architecture for Sk6.0 — a full-stack lottery platform targeting Web, Android, and iOS users. The architecture is designed from day one to support 1,000+ concurrent users with sub-2-second response times at peak load (1 PM and 3 PM draws).

1.1 Design Goals

Goal	Target	How We Achieve It
Scalability	1,000+ concurrent users	Horizontal replicas + async I/O + connection pooling
Performance	< 2s response at peak	Redis caching + DB indexes + CDN for assets
Reliability	99.5% uptime	Multiple replicas + health checks + auto-restart
Security	Zero critical bugs	JWT + RBAC + server-side validation + audit log
Maintainability	Layered, testable code	Clean Architecture + dependency injection
Development Speed	Deliver by 30 June	Parallel backend/mobile tracks + automated testing

1.2 Non-Functional Requirements

Category	Requirement	Measurement
Throughput	Handle 300+ requests per second at peak	k6 / Locust load test
Latency	p95 response time under 800ms for API calls	Prometheus histograms
Availability	Target 99.5% uptime over each month	Uptime monitor
Data Safety	Zero wallet balance inconsistencies	Audit trail + DB constraints
Mobile UX	App cold start under 3 seconds	Flutter performance profile
Mobile Size	APK size under 25 MB	Build output check

2. High-Level System Architecture

The system is divided into four layers: Clients, Edge Layer (Nginx), Application Tier (FastAPI replicas), and Data Tier (PostgreSQL + Redis). Background workers (Celery) run as a separate tier.

2.1 System Component Diagram



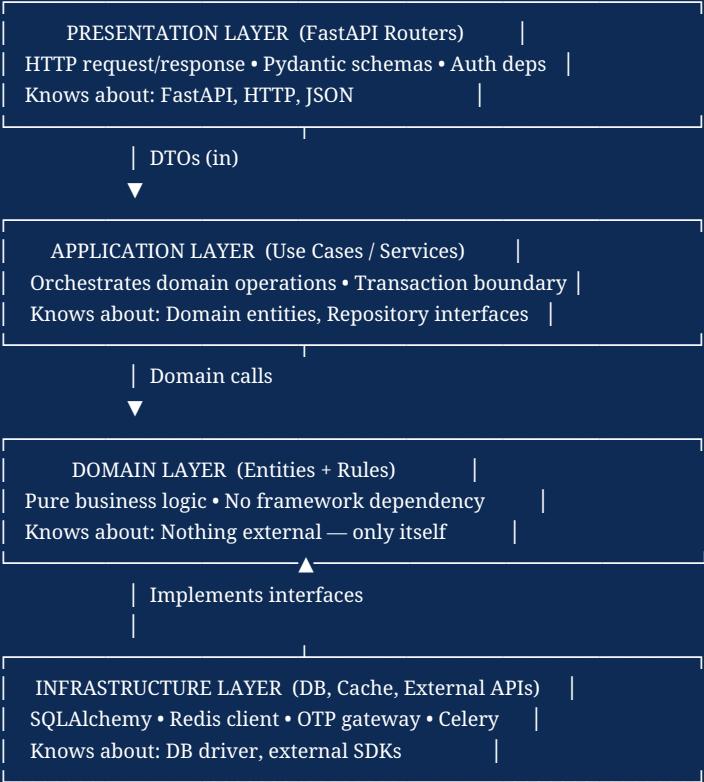
2.2 Component Responsibilities

Component	Technology	Responsibility
Mobile / Web Client	Flutter 3.x	UI rendering, user input, local caching, API calls
Gateway	Nginx 1.25	TLS termination, rate limiting, load balancing, static asset serving
Application Servers	FastAPI + uvicorn	HTTP handlers, request validation, business logic orchestration
Database Pool	PgBouncer	Multiplex 1,200 app connections into ~50 Postgres connections
Primary Database	PostgreSQL 16	Durable storage for users, wallets, bets, results, audit log
Cache / Session Store	Redis 7	Hot data cache, JWT revocation list, session state, Celery broker
Background Workers	Celery + Beat	Draw auto-close, result settlement, report generation, notifications
Observability	Prometheus + Grafana	Metrics collection, dashboards, alerting on SLO breach

3. Backend — Clean Architecture

We follow Clean Architecture (hexagonal / onion style) to separate business logic from framework and infrastructure concerns. This makes the code testable, maintainable, and replaceable at any layer.

3.1 The Four Layers



DEPENDENCY DIRECTION: Outer layers depend on Inner, never the reverse. Domain has zero imports.

3.2 Layer Responsibilities

Layer	What It Contains	What It Must Never Do
Presentation	FastAPI routers, Pydantic request/response schemas, auth middleware, exception handlers	Contain business rules or database queries
Application	Use-case classes (CreateRechargeUseCase, ApproveWithdrawalUseCase), DTOs, transaction coordination	Import FastAPI, HTTP concepts, or SQL
Domain	Pure Python dataclasses for entities (User, Wallet, Bet), domain exceptions, invariant checks	Import anything from infrastructure, app, or presentation
Infrastructure	SQLAlchemy models, repository implementations, Redis clients, external API adapters, Celery tasks	Contain business rules — it only persists and fetches

3.3 Example — Recharge Flow Across All Layers

Here is how a single user recharge request flows from HTTP to database, passing through every layer:

```
# 1. PRESENTATION — app/api/v1/recharge.py
@router.post('/request', status_code=201)
async def submit_recharge(
    payload: RechargeRequestSchema,      # Pydantic validates input
    current_user: User = Depends(get_current_user),
    uc: CreateRechargeUseCase = Depends(create_recharge_use_case),
):
    result = await uc.execute(
        user_id=current_user.id,
        amount=payload.amount,
        payment_ref=payload.payment_ref,
    )
    return RechargeResponse.from_domain(result)

# 2. APPLICATION — app/application/use_cases/create_recharge.py
class CreateRechargeUseCase:
    def __init__(
        self,
        recharge_repo: IRechargeRepository,    # Interface from domain
        user_repo: IUserRepository,
        uow: IUnitOfWork,                      # Transaction scope
    ):
        self._recharge_repo = recharge_repo
        self._user_repo = user_repo
        self._uow = uow

    async def execute(self, user_id, amount, payment_ref):
        async with self._uow:
            # Domain logic enforces business rules
            request = RechargeRequest.create(
                user_id=user_id,
                amount=amount,
                payment_ref=payment_ref,
            ) # raises DomainError if invalid

            # Check duplicate via repository
            if await self._recharge_repo.exists_by_ref(payment_ref):
                raise DuplicateTransactionError()

            await self._recharge_repo.add(request)
            await self._uow.commit()
            return request

# 3. DOMAIN — app/domain/entities/recharge.py
@dataclass(frozen=True)
class RechargeRequest:
```

```

id: UUID
user_id: UUID
amount: Decimal
payment_ref: str
status: RechargeStatus
created_at: datetime

@classmethod
def create(cls, user_id, amount, payment_ref):
    if amount <= 0 or amount > 9999:
        raise DomainError('Amount must be 1..9999')
    if not re.match(r'^\d{1,12}$', payment_ref):
        raise DomainError('Payment ref must be 1..12 digits')
    return cls(
        id=uuid4(), user_id=user_id, amount=amount,
        payment_ref=payment_ref,
        status=RechargeStatus.PENDING,
        created_at=datetime.utcnow(),
    )

# 4. INFRASTRUCTURE — app/infrastructure/db/repositories/recharge.py
class SQLAlchemyRechargeRepository(IRechargeRepository):
    def __init__(self, session: AsyncSession):
        self._s = session

    async def add(self, entity: RechargeRequest) -> None:
        row = RechargeRow.from_domain(entity) # map to ORM model
        self._s.add(row)

    async def exists_by_ref(self, payment_ref: str) -> bool:
        stmt = select(RechargeRow).where(
            RechargeRow.payment_ref == payment_ref,
            RechargeRow.status != 'rejected',
        )
        return (await self._s.execute(stmt)).first() is not None

```

Why this pattern matters: If we ever switch from PostgreSQL to another database, only the Infrastructure layer changes. Domain and Application layers are untouched. This also makes unit testing trivial — inject a fake repository and test the use case in isolation.

4. Backend — Folder Structure & Code Layout

The backend codebase follows a feature-aligned Clean Architecture layout. Each feature lives in its own use-case module but shares the infrastructure and domain primitives.

```
backend/
├── app/
│   ├── main.py          # FastAPI entry point, router registration
│   └── core/            # CROSS-CUTTING CONCERNS
│       ├── config.py    # Settings (pydantic-settings)
│       ├── security.py  # JWT helpers, password hashing
│       ├── exceptions.py # Global error handlers
│       ├── logging.py   # structlog configuration
│       └── container.py  # Dependency injection wiring
├── api/                 # PRESENTATION LAYER
│   ├── deps.py          # Shared FastAPI dependencies
│   └── v1/
│       ├── auth.py      # /auth/register, /auth/login
│       ├── users.py     # /users/me, /users/me/bets
│       ├── wallet.py    # /wallet/balance, /wallet/history
│       ├── recharge.py  # /recharge/request, /recharge/history
│       ├── withdrawal.py # /withdrawal/request
│       ├── draws.py     # /draws/today, /draws/open
│       ├── bets.py      # /bets (place, cart)
│       ├── results.py   # /results
│       ├── announcements.py # /announcements
│       └── admin/       # Admin-only routes (require admin JWT)
│           ├── users.py
│           ├── recharge.py
│           ├── withdrawal.py
│           ├── results.py
│           ├── reports.py
│           └── announcements.py
├── schemas/             # PRESENTATION DTOs (Pydantic)
│   ├── auth.py
│   ├── wallet.py
│   ├── recharge.py
│   └── ...
├── application/         # USE-CASE LAYER
│   ├── interfaces/      # Abstract repository/service interfaces
│   │   ├── repositories.py
│   │   ├── unit_of_work.py
│   │   └── services.py
│   └── use_cases/
│       └── auth/        # register, login, reset password
```

```

├── wallet/      # get_balance, list_transactions
├── recharge/    # create, approve, reject
├── withdrawal/  # create, approve, reject
├── betting/     # add_to_cart, confirm_purchase
├── results/     # enter, verify, publish, settle
├── admin/       # approve_signup, block, reset_wallet
├── reports/     # sales, cost, pnl
├── domain/      # DOMAIN LAYER (zero deps)
├── entities/    # Pure dataclasses
│   ├── user.py
│   ├── wallet.py
│   ├── recharge.py
│   ├── bet.py
│   ├── draw.py
│   └── result.py
├── value_objects/ # Phone, Money, TicketNumber
├── events/       # Domain events (BetPlaced, ResultPublished)
├── exceptions.py # DomainError, InvariantViolation
├── infrastructure/ # INFRASTRUCTURE LAYER
│   ├── db/
│   │   ├── engine.py # SQLAlchemy async engine
│   │   ├── session.py # Session factory
│   │   ├── uow.py     # UnitOfWork implementation
│   │   ├── models/    # SQLAlchemy ORM models (mapped to tables)
│   │   └── repositories/ # Concrete repository implementations
│   ├── cache/
│   │   ├── redis_client.py
│   │   └── cache_service.py
│   ├── external/
│   │   ├── otp_gateway.py # zplusone.in WhatsApp API
│   │   └── pdf_generator.py # ReportLab for PDF reports
│   ├── tasks/         # Celery
│   │   ├── celery_app.py
│   │   ├── scheduler.py # Beat schedule
│   │   ├── draws.py     # close_draw, settle_results
│   │   ├── notifications.py
│   │   └── reports.py
│   ├── migrations/    # Alembic migrations
│   ├── env.py
│   └── versions/
├── tests/
│   ├── unit/          # Use-case tests with fake repos
│   ├── integration/   # DB + API integration tests
│   └── load/          # Locust scripts
├── docker/
└── Dockerfile

```

```
| └─ entrypoint.sh  
|  
└─ docker-compose.yml  
└─ pyproject.toml  
└─ alembic.ini
```

Dependency injection is done in `app/core/container.py`. FastAPI's `Depends()` resolves the concrete implementation at request time, but the routers only see interfaces.

5. Database Design & Transaction Strategy

5.1 Schema Highlights

The full DDL is in backend/migrations/schema.sql. Critical tables and the reason each matters:

Table	Purpose	Key Indexes
users	User accounts, status (pending/active/blocked)	phone (unique), status, last_login_at
wallets	Wallet balance (1:1 with users) — isolated row	user_id (unique)
sessions	JWT session tracking for force-logout	token_hash (unique), (user_id, is_active)
transactions	Ledger of every money movement	user_id, type, created_at DESC
recharge_requests	Recharge lifecycle — pending/approved/rejected	payment_ref (unique where !=rejected), status
withdrawal_requests	Withdrawal approvals queue	user_id, status, created_at DESC
draws	One row per scheduled draw instance	(schedule_id, draw_date) unique, status
bets	User bet tickets	(draw_id, ticket_number), user_id
results	Published winning numbers	draw_id (unique), published_at
winners	Reverse-index from result to winning bets	user_id, draw_id
audit_logs	Immutable record of every admin action	actor_id, action, entity_id, created_at

5.2 Wallet Transaction Safety

Wallet balance changes are the most critical operation. Every credit/debit must be atomic and consistent. We use a combination of database-level constraints and application-level patterns:

- **Transaction Scope:** All balance changes happen inside a database transaction (SERIALIZABLE isolation for money operations).
- **Row Locking:** SELECT ... FOR UPDATE on the wallet row before any debit — prevents concurrent double-spend.
- **DB Constraint:** CHECK (balance >= 0) in the schema — double safety net. Negative balance rejected by the database itself.
- **Audit Trail:** Every tx writes to transactions table with balance_before and balance_after snapshots for audit.
- **Duplicate Check:** Unique index on (user_id, type, payment_ref) where type='recharge' prevents duplicate credits.

```
# Example: safe wallet debit pattern
async def debit_wallet(session, user_id, amount):
    # 1. Lock the wallet row
    wallet = (await session.execute(
        select(Wallet).where(Wallet.user_id == user_id).with_for_update()
    )).scalar_one()

    # 2. Check balance
    if wallet.balance < amount:
        raise InsufficientFundsError()

    # 3. Record snapshot, update, commit
    balance_before = wallet.balance
    wallet.balance -= amount
    session.add(Transaction(
```

```
    user_id=user_id, amount=amount, type='bet',
    balance_before=balance_before,
    balance_after=wallet.balance,
))
await session.commit()
```

5.3 Connection Pooling Configuration

Setting	Value	Rationale
PgBouncer pool_mode	transaction	Best for short, stateless queries. Each transaction gets a fresh server connection.
PgBouncer max_client_conn	1,500	Headroom above 1,200 target concurrent users.
PgBouncer default_pool_size	25	~25 active Postgres connections per PgBouncer instance.
Postgres max_connections	100	Never exceed this — PgBouncer stops opening new connections at 50.
SQLAlchemy pool_size	10 per replica	3 replicas × 10 = 30 app-side connections, pooled by PgBouncer.
SQLAlchemy pool_recycle	3,600 sec	Recycle stale connections every hour.

6. API Design Standards

6.1 Conventions

Topic	Standard
URL scheme	All API under /api/v1/ — never break a version. Add /v2/ for breaking changes.
Method semantics	GET (read), POST (create/action), PUT (full update), PATCH (partial), DELETE (remove)
Resource naming	Plural nouns — /users, /bets, /draws. Actions as sub-paths: /draws/{id}/close
Authentication	Bearer JWT in Authorization header. 401 if missing/expired, 403 if lacking role.
Content type	application/json for requests and responses. Multipart only for file uploads (recharge screenshot).
Pagination	Cursor-based for large lists: ?cursor=abc&limit=20. Page-based only for admin reports.
Dates	ISO-8601 with timezone (UTC stored, rendered as Asia/Kolkata in admin panel).
Money	Integer paise in API payload (e.g. 50000 = ₹500.00). Never use floats.
Idempotency	Critical POST endpoints (recharge, withdrawal) accept Idempotency-Key header.
Versioning	All clients pin to /api/v1. Mobile app checks /health and handles old-version warning.

6.2 Standard Error Response

HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json

```
{
  "error": {
    "code": "DUPLICATE_TRANSACTION_ID",
    "message": "This transaction ID has already been used.",
    "details": {
      "payment_ref": "123456789012"
    },
    "request_id": "req-a1b2c3d4-e5f6"
  }
}
```

6.3 Rate Limiting

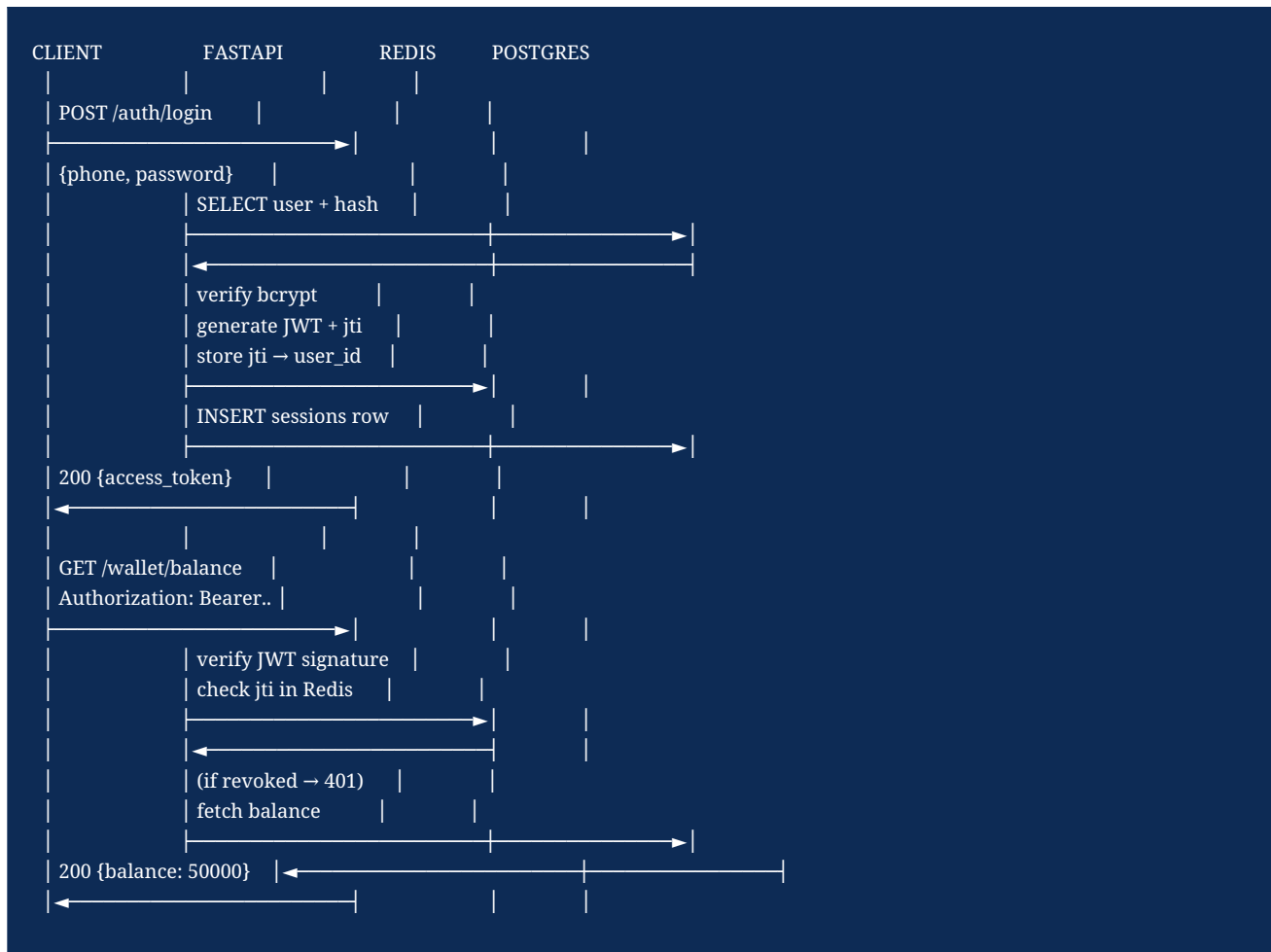
Endpoint Group	Limit	Reason
/auth/login	10 / min / IP	Prevent brute force
/auth/register	5 / hour / IP	Prevent spam signups
/auth/otp	3 / 10 min / phone	Prevent OTP abuse
/recharge/request	10 / hour / user	Prevent duplicate submissions
All other authed	300 / min / user	Generous for normal use, blocks abuse
Admin endpoints	60 / min / admin	Reasonable for admin panel usage

6.4 OpenAPI Documentation

FastAPI generates OpenAPI 3 docs automatically at `/api/docs` (Swagger UI) and `/api/redoc`. Both are disabled in production and only available behind admin auth. The OpenAPI JSON is exported to the mobile team for automatic client SDK generation.

7. Authentication & Authorization

7.1 JWT Flow



7.2 Token Structure

JWT Payload (HS256):

```
{
  "sub": "7a4e...",    # user UUID
  "role": "user",      # 'user' | 'admin' | 'superadmin'
  "jti": "b8c3...",    # session ID (used for revocation)
  "iat": 1712345678,   # issued at
  "exp": 1712432078    # 24 hours from issue
}
```

Stored in Redis:

Key: session:{jti}

Value: {user_id, role, issued_at, status}

TTL: matches JWT exp

7.3 Role-Based Access Control

Route Pattern	Allowed Roles	Enforcement
/api/v1/auth/*	Public	No auth required
/api/v1/users/me	user, admin, superadmin	Valid JWT required
/api/v1/wallet/*	user, admin, superadmin	Valid JWT required
/api/v1/bets/*	user (not blocked)	Valid JWT + status check
/api/v1/admin/*	admin, superadmin	Valid JWT + role check
/api/v1/admin/users/{id}/block	admin, superadmin	Action logged to audit_logs
/api/v1/admin/system/*	superadmin only	Only root admin can change system config

7.4 Block User – Immediate Logout

When admin blocks a user, we need to invalidate all active sessions within seconds. Flow:

```
async def block_user(user_id):
    async with uow:
        # 1. Set status = blocked in DB
        user = await user_repo.get(user_id)
        user.status = UserStatus.BLOCKED
        await user_repo.save(user)

        # 2. Revoke all active sessions — mark in DB
        sessions = await session_repo.list_active(user_id)
        for s in sessions:
            s.is_active = False
            await session_repo.save(s)

        # 3. Invalidate Redis entries — instant effect on next request
        for s in sessions:
            await redis.delete(f'session:{s.token_hash}')

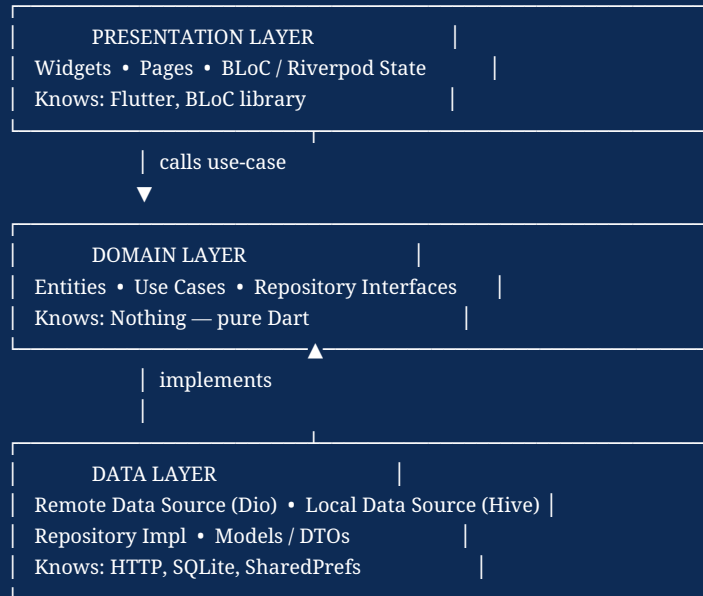
        # 4. Write audit log
        await audit_log.record('user_block', user_id, actor=current_admin.id)
        await uow.commit()
```

Result: The blocked user's very next API call fails with 403. Their Flutter app detects 403 and immediately navigates to the login screen with an 'Account Blocked' message. Force-logout effective within 1 request.

8. Mobile App — Clean Architecture (Flutter)

The Flutter app follows the same Clean Architecture philosophy as the backend, adapted for a UI application. Features are isolated; each has its own Data, Domain, and Presentation layers.

8.1 Mobile Layer Diagram



8.2 Mobile Folder Structure

```
mobile/
├── lib/
│   ├── main.dart
│   ├── app.dart          # MaterialApp, routing, theme
│   └── core/             # SHARED CROSS-CUTTING
│       ├── constants/
│       │   ├── api_endpoints.dart
│       │   └── app_constants.dart
│       ├── errors/
│       │   ├── failures.dart  # Domain-level failure types
│       │   └── exceptions.dart # Low-level exceptions
│       ├── network/
│       │   ├── dio_client.dart # HTTP client with interceptors
│       │   ├── auth_interceptor.dart
│       │   └── retry_interceptor.dart
│       └── storage/
│           ├── secure_storage.dart # Token storage (flutter_secure_storage)
│           └── cache_storage.dart # Hive for offline cache
```

```

└─ theme/
  │ └─ app_colors.dart
  │ └─ app_theme.dart
└─ router/
  │ └─ app_router.dart # go_router configuration
└─ di/
  │ └─ injection.dart # get_it service locator
└─ features/ # FEATURE MODULES
  │ └─ auth/
  │ │ └─ data/
  │ │ │ └─ datasources/
  │ │ │ │ └─ auth_remote_datasource.dart
  │ │ │ │ └─ auth_local_datasource.dart
  │ │ │ └─ models/
  │ │ │ │ └─ user_model.dart # fromJson/toJson
  │ │ │ │ └─ login_response.dart
  │ │ │ └─ repositories/
  │ │ │ │ └─ auth_repository_impl.dart
  │ │ └─ domain/
  │ │ │ └─ entities/
  │ │ │ │ └─ user.dart # pure Dart class
  │ │ │ └─ repositories/
  │ │ │ │ └─ auth_repository.dart # abstract
  │ │ │ └─ usecases/
  │ │ │ │ └─ login.dart
  │ │ │ │ └─ register.dart
  │ │ │ │ └─ reset_password.dart
  │ │ └─ presentation/
  │ │ │ └─ bloc/
  │ │ │ │ └─ auth_bloc.dart
  │ │ │ │ └─ auth_event.dart
  │ │ │ │ └─ auth_state.dart
  │ │ │ └─ pages/
  │ │ │ │ └─ login_page.dart
  │ │ │ │ └─ signup_page.dart
  │ │ │ │ └─ reset_password_page.dart
  │ │ │ └─ widgets/
  │ │ │ │ └─ phone_input.dart
  │ │ │ │ └─ otp_field.dart
  │ └─ wallet/ # Same structure
  └─ recharge/
  └─ withdrawal/
  └─ lottery/
  └─ cart/
  └─ results/
  └─ notifications/
  └─ profile/
└─ shared/ # SHARED UI

```

```

├── widgets/
│   ├── app_button.dart
│   ├── loading_indicator.dart
│   └── error_view.dart
└── extensions/
    └── context_extensions.dart

test/
├── unit/
├── widget/
└── integration/

android/
ios/
pubspec.yaml

```

9. Mobile App — State Management & Offline Strategy

9.1 State Management — BLoC Pattern

We use `flutter_bloc` for predictable state. Each feature has its own Bloc. The UI dispatches Events, the Bloc emits States, widgets rebuild from those States.

```
// Event (user intent)
sealed class AuthEvent {}
class LoginRequested extends AuthEvent {
  final String phone;
  final String password;
  LoginRequested({required this.phone, required this.password});
}

// State (what to render)
sealed class AuthState {}
class AuthInitial extends AuthState {}
class AuthLoading extends AuthState {}
class AuthSuccess extends AuthState { final User user; ... }
class AuthFailure extends AuthState { final String message; ... }
class AuthBlocked extends AuthState {}

// Bloc (business glue)
class AuthBloc extends Bloc<AuthEvent, AuthState> {
  final LoginUseCase _login;
  AuthBloc(this._login) : super(AuthInitial()) {
    on<LoginRequested>(_onLoginRequested);
  }

  Future<void> _onLoginRequested(
    LoginRequested event,
    Emitter<AuthState> emit,
  ) async {
    emit(AuthLoading());
    final result = await _login(event.phone, event.password);
    result.fold(
      (failure) => emit(AuthFailure(failure.message)),
      (user) => emit(AuthSuccess(user)),
    );
  }
}

// Widget (UI)
BlocBuilder<AuthBloc, AuthState>(
  builder: (context, state) {
    return switch (state) {
      AuthLoading() => const LoadingIndicator(),
      AuthFailure(:final message) => ErrorView(message: message),
```

```

    AuthBlocked() => const BlockedView(),
    _ => const LoginForm(),
  };
},
)

```

9.2 Offline & Caching Strategy

Data Type	Cache Strategy	TTL / Invalidation
User profile	Hive (encrypted box)	Refresh on app open
Wallet balance	In-memory (BLoC state)	Refresh before each bet placement
Open draws	Hive + 60s TTL	Refresh when expired or pull-to-refresh
Results (published)	Hive (permanent)	Immutable once published
Bet history	Hive + paginated	Append new, keep last 100 locally
Announcements	Hive + refresh on open	Refresh on app foreground
JWT token	flutter_secure_storage (KeyStore)	Cleared on logout or 401

9.3 Offline UX Rules

- **Stale-while-revalidate:** Show cached data immediately — refresh in background with skeleton.
- **No Bet Offline:** Cannot place bets — UI disabled with 'You are offline' banner.
- **No Wallet Actions Offline:** Cannot recharge or withdraw — block submit until connection returns.
- **Retry Queue:** Queued locally (Hive), submitted when back online (optional post-launch).

9.4 Network Layer — Dio with Interceptors

```

class DioClient {
  final Dio dio;

  DioClient() : dio = Dio(BaseOptions(
    baseUrl: ApiEndpoints.baseUrl,
    connectTimeout: const Duration(seconds: 10),
    receiveTimeout: const Duration(seconds: 30),
  )) {
    dio.interceptors.addAll([
      AuthInterceptor(),    // attach Bearer token
      RetryInterceptor(),    // exponential backoff on 5xx
      LoggingInterceptor(),  // redact sensitive fields
      ErrorMappingInterceptor(), // map HTTP → Failure
    ]);
  }
}

```

10. Scaling Strategy — Handling 1,000+ Concurrent Users

Concurrent user count is misleading on its own — what matters is requests per second and how long each request holds resources. At 1,000 concurrent users, we estimate:

Metric	Estimate	Notes
Peak concurrent users	1,000–1,200	During 3 PM draw close window
Requests per user per min	~6 req/min	Page views + poll + actions
Peak RPS	~100–200	With bursts up to 300 at T-60 seconds before draw close
DB queries per request	~2–5 avg, ~10 at bet	With Redis cache hit rate > 80%
Peak DB QPS	~500–800	Comfortable within Postgres limits on a 4-core VM

10.1 Horizontal Scaling Tiers

Tier	Scaling Method	Resource Plan
Nginx	Vertical (single instance sufficient up to 10k req/s)	2 vCPU, 2 GB RAM
FastAPI	Horizontal — 3 replicas behind Nginx, uvicorn 2 workers each	3 containers × 2 vCPU × 2 GB
PgBouncer	Horizontal — 2 replicas share pool	2 containers × 1 vCPU × 1 GB
PostgreSQL	Vertical first — 4 vCPU, 8 GB. Read replica when needed.	Single node, upgradable
Redis	Single instance initially; Redis Sentinel for HA later	2 vCPU, 2 GB RAM
Celery Worker	Horizontal — 2 workers	2 containers × 1 vCPU × 1 GB

10.2 What Prevents the System from Falling Over

Failure Mode	Mitigation
Thundering herd at draw close	Redis caches 'draws open' state — 95% of reads skip DB. Client polling at exponential backoff.
Database connection exhaustion	PgBouncer caps Postgres connections at 50. Extra app connections wait in pool queue.
Slow query blocks replicas	Statement timeout = 5 seconds. Long-running reports run on Celery, not API.
One replica crashes	Nginx health check removes it. Docker auto-restarts. Users transparently routed to other replicas.
Redis down	API degrades gracefully — skips cache, hits DB directly. Sessions fall back to DB lookup.
Celery worker crash	Tasks auto-retry up to 3 times. Settlement tasks are idempotent.

10.3 Growth Path

Stage	Users	What Changes
Launch	0 – 2,000	Current architecture unchanged
Growth Phase	2,000 – 5,000	Scale FastAPI replicas from 3 – 5. No code change.
At Target	5,000 – 10,000	Add read replica for reports. Move heavy reports to data warehouse.
Beyond	10,000+	Shard hot tables (bets) by date. Move to managed Postgres.

11. Caching Strategy (Redis)

Cache Key	Data	TTL	Invalidation
draws:today	List of today's draws + status	60s	On draw status change
draws:open	Currently-open draw IDs	30s	On draw close event
result:{draw_id}	Published result	permanent	Never — immutable
user:{id}:wallet	Wallet balance	10s	On any wallet mutation
user:{id}:profile	User profile	5 min	On user update
system:operational	Is platform open today?	5 min	On holiday create/delete
announcement:active	Active announcements	2 min	On announcement create/delete
session:{jti}	Session metadata for JWT check	24h	On logout or block
rate_limit:{user}:{bucket}	Rate limit counter	60s	Auto (Redis expire)

11.1 Cache Patterns

- **Cache-aside:** Read from cache. Miss → fetch from DB → populate cache → return. Most common pattern.
- **Write-through:** Update DB. Immediately invalidate cache key. Next read re-populates. Used for wallet, profile.
- **Single-flight:** On cache miss during high-load event (draw close), use single-flight so only one request hits DB per key.

12. Background Jobs (Celery)

Celery handles all heavy, deferred, or scheduled work so the API stays fast. Redis acts as the message broker.

12.1 Scheduled Tasks (Celery Beat)

Task	Schedule	Purpose
close_draw('dear3_1pm')	12:55 daily	Lock betting, set status → result_pending
close_draw('dear3_3pm')	14:55 daily	Lock betting
close_draw('dear3_6pm')	17:55 daily	Lock betting
close_draw('dear3_8pm')	19:55 daily	Lock betting
rollover_draws	00:05 daily	Create today's draw instances from schedule templates
cleanup_expired_sessions	Every hour	Remove sessions past expiry from DB (Redis auto-expires)
generate_daily_reports	22:00 daily	Pre-compute sales/cost/P&L for the day, store in reports table

12.2 On-Demand Tasks

Task	Triggered By	Purpose
settle_draw_result(draw_id)	Admin publishes result	Compute winners, credit wallets, send notifications
send_notification(user_id)	Recharge approved / bet won	Push notification via FCM
send_otp(phone, code)	Signup / password reset	Call zplusone WhatsApp API
generate_pdf_report(params)	Admin clicks PDF download	Render report → signed URL → email link

12.3 Settlement Task Pattern

```
@celery_app.task(
    bind=True,
    max_retries=3,
    default_retry_delay=60,
    acks_late=True, # ack only after success
)
def settle_draw_result(self, draw_id: str):
    try:
        with get_sync_session() as session:
            # 1. Fetch draw + result
            draw = session.get(Draw, draw_id)
            result = session.query(Result).filter_by(draw_id=draw_id).one()

            # 2. Find all winning bets (ticket matches result)
            winning_bets = session.query(Bet).filter(
                Bet.draw_id == draw_id,
                Bet.ticket_number == result.winning_number,
                Bet.status == BetStatus.PLACED,
            ).all()

            # 3. For each winner: credit wallet + record winner row
```

```
for bet in winning_bets:
    payout = calculate_payout(bet, result)
    credit_wallet(session, bet.user_id, payout)
    session.add(Winner(bet_id=bet.id, ...))
    bet.status = BetStatus.WON

# 4. Update losing bets
session.query(Bet).filter(
    Bet.draw_id == draw_id,
    Bet.ticket_number != result.winning_number,
    Bet.status == BetStatus.PLACED,
).update({'status': BetStatus.LOST})

# 5. Mark draw as settled
draw.status = DrawStatus.RESULT_PUBLISHED
session.commit()

# 6. Fan-out notifications (separate tasks for parallelism)
for bet in winning_bets:
    send_notification.delay(bet.user_id, 'You won!')

except Exception as exc:
    raise self.retry(exc=exc)
```

The task is idempotent — if it runs twice (retry), it re-queries by status='placed' and skips already-settled bets. Wallet credits are safe because they go through `credit_wallet()` which logs to transactions.

13. Security Architecture

Layer	Controls
Transport	TLS 1.3 only. HSTS header. Certificate from Let's Encrypt, auto-renewed.
Nginx Edge	Rate limiting, bot detection via user-agent heuristics, max request body size (10 MB).
Application	JWT with HS256, 24-hour expiry, Redis-backed revocation list. Role-based access on every admin route. Pydantic validates every input. SQLAlchemy parametrized queries prevent SQL injection.
Data	Passwords hashed with bcrypt (cost=12). Sensitive PII (bank numbers) stored with pgcrypto-encrypted columns. Database backups encrypted at rest.
Mobile	JWT stored in Keystore (Android) / Keychain (iOS) via flutter_secure_storage. Certificate pinning in production builds. Root/jailbreak detection warning.
Admin	Separate admin app. Admin JWT with 2-hour expiry (shorter than user). Every admin action logged to audit_logs with actor_id, old/new values, IP.
Input Validation	Server-side enforcement of: recharge amount ≤ 9999 , payment_ref numeric ≤ 12 digits, duplicate payment_ref rejected, ticket number numeric only (3 or 4 digits).
Rate Limiting	Per-IP and per-user limits on all auth endpoints. OTP generation capped at 3 per 10 minutes per phone.
CSRF	Not applicable for pure API with Bearer tokens — but admin web uses SameSite=strict cookies if any web-based admin is added.
Audit	Every mutation to wallet, user status, result, or holiday is logged. Audit log is append-only — no updates or deletes permitted.

13.1 OWASP Top 10 Checklist

OWASP Risk	Mitigation
A01: Broken Access Control	RBAC enforced on every admin route via Depends(require_admin)
A02: Cryptographic Failures	TLS 1.3, bcrypt, no plaintext secrets anywhere
A03: Injection	SQLAlchemy parametrized queries, Pydantic type validation
A04: Insecure Design	Threat-modelled flows for wallet, result, auth
A05: Misconfiguration	Production Dockerfile hardened, no debug mode, secrets via env vars
A06: Vulnerable Components	Dependabot on GitHub, weekly scan of pip/pub deps
A07: Identification & Auth	JWT + Redis revocation, rate-limited login, bcrypt
A08: Data Integrity	DB CHECK constraints, audit log, wallet snapshots
A09: Logging Failures	Structured JSON logs, admin actions to audit table
A10: SSRF	No user-supplied URLs fetched by backend

14. Observability — Logs, Metrics, Alerts

14.1 Logging

- **Format:** JSON format, one event per line, structlog library.
- **Fields always present:** request_id, user_id, endpoint, duration_ms, status, ip.
- **Redacted fields:** password, token, payment_ref (shown masked).
- **Destination:** Docker captures stdout. Shipped to Loki or CloudWatch in production.

14.2 Metrics (Prometheus)

Metric	Type	What It Tells Us
http_request_duration_seconds	Histogram	p50/p95/p99 latency per endpoint
http_requests_total	Counter	Request volume by status code
db_query_duration_seconds	Histogram	Slow query detection
celery_task_duration_seconds	Histogram	Background job performance
cache_hit_ratio	Gauge	Effectiveness of Redis caching
active_sessions	Gauge	Current logged-in user count
bet_placement_total	Counter	Business metric — bets per draw
wallet_balance_inconsistency	Counter	Must remain zero — alert if increments

14.3 Dashboards (Grafana)

- **System Health:** Request rate, latency p95, error rate, active users.
- **Business:** Bets per draw, sales per hour, recharge approval rate.
- **Database:** Connection pool usage, slow queries, deadlocks.
- **Redis:** Cache hit rate, memory usage, slow commands.

14.4 Alerts

Alert	Condition	Severity
High error rate	5xx rate > 1% for 5 min	PAGE
Latency spike	p95 > 2s for 5 min	PAGE
Database down	Any replica fails health check	PAGE
Redis down	Redis unreachable for 1 min	PAGE
Celery backlog	Queue length > 1000 for 5 min	WARN
Settlement failure	settle_draw task retried 3 times	PAGE
Wallet mismatch	wallet_balance_inconsistency > 0	PAGE (CRITICAL)
Disk usage	Postgres disk > 80%	WARN

15. Deployment Architecture

15.1 Production Environment

Service	Replicas	CPU / RAM	Ports
Nginx	1	2 vCPU / 2 GB	80, 443
FastAPI	3	2 vCPU / 2 GB ea.	8000 (internal)
PgBouncer	1	1 vCPU / 1 GB	6432 (internal)
PostgreSQL	1 primary	4 vCPU / 8 GB	5432 (internal)
Redis	1	2 vCPU / 2 GB	6379 (internal)
Celery Worker	2	1 vCPU / 1 GB ea.	—
Celery Beat	1	0.5 vCPU / 512 MB	—
Prometheus	1	1 vCPU / 2 GB	9090 (internal)
Grafana	1	1 vCPU / 1 GB	3000 (admin-only)

Total: ~16 vCPU, ~28 GB RAM. Fits comfortably on a single 4-socket VM or spread across 2 mid-size VMs for high availability.

15.2 docker-compose.yml (Production)

```
version: '3.9'
services:
  nginx:
    image: nginx:1.25-alpine
    ports: ['80:80', '443:443']
    volumes: ['./nginx:/etc/nginx/conf.d:ro', './certs:/etc/letsencrypt:ro']
    depends_on: [api]

  api:
    build: ./backend
    command: gunicorn app.main:app -w 2 -k uvicorn.workers.UvicornWorker
      --bind 0.0.0.0:8000 --max-requests 1000 --timeout 30
    environment:
      DATABASE_URL: postgresql+asyncpg://sk6:***@pgbouncer:6432/sk6
      REDIS_URL: redis://redis:6379/0
      SECRET_KEY: ${SECRET_KEY}
    deploy:
      replicas: 3
      resources:
        limits: { cpus: '2', memory: 2G }
    depends_on: [pgbouncer, redis]

  pgbouncer:
    image: edoburu/pgbouncer:1.21
    environment:
      DB_HOST: postgres
      POOL_MODE: transaction
      MAX_CLIENT_CONN: 1500
      DEFAULT_POOL_SIZE: 25

  postgres:
```

image: postgres:16-alpine
environment:
 POSTGRES_DB: sk6
 POSTGRES_PASSWORD: \${DB_PASSWORD}
volumes: ['pgdata:/var/lib/postgresql/data']
command: postgres -c max_connections=100 -c shared_buffers=2GB

redis:
 image: redis:7-alpine
 command: redis-server --maxmemory 1gb --maxmemory-policy allkeys-lru
 volumes: ['redisdata:/data']

celery-worker:
 build: ./backend
 command: celery -A app.infrastructure.tasks.celery_app worker --loglevel=info --concurrency=4
 deploy: { replicas: 2 }
 depends_on: [redis, postgres]

celery-beat:
 build: ./backend
 command: celery -A app.infrastructure.tasks.celery_app beat --loglevel=info
 depends_on: [redis]

volumes:
 pgdata:
 redisdata:

16. Development Workflow & CI/CD

16.1 Git Flow

Branch	Purpose	Deploy Target
main	Stable, production-ready	Production
develop	Integration of all features	Staging
feature/{ticket-id}	Individual feature work	Preview (PR deploy)
hotfix/{ticket-id}	Critical production fixes	Production after review

16.2 Pull Request Rules

- All tests pass (unit + integration).
- Code coverage does not drop below 75%.
- At least 1 reviewer approval (2 for admin/security code).
- No Typescript/Python linter errors.
- Database migrations reviewed separately.
- Changelog entry for user-facing changes.

16.3 CI Pipeline (GitHub Actions)

```
# .github/workflows/backend-ci.yml
on: [pull_request]
jobs:
  test:
    steps:
      - checkout
      - setup-python 3.12
      - pip install -r backend/requirements.txt
      - ruff check backend/
      - mypy backend/app/
      - pytest backend/tests/unit/ --cov=app --cov-fail-under=75
      - pytest backend/tests/integration/

  security-scan:
    steps:
      - bandit -r backend/app/
      - safety check -r backend/requirements.txt

  build-image:
    needs: [test, security-scan]
    steps:
      - docker build -t sk6-api:${{ github.sha }} ./backend
      - push to registry
```

16.4 Release Cadence

Release	Frequency	Content
Backend	Every merge to develop → staging.	API fixes, features

	Tagged versions — production weekly.	
Mobile	Bi-weekly release candidates on TestFlight / internal track.	New screens, bug fixes
Hotfix	As needed — same-day deploy for production issues.	Critical fixes only

17. Performance Targets & Load Testing

17.1 SLO Targets

Endpoint Class	p50	p95	p99
Auth (login, register)	150 ms	500 ms	1 s
Wallet balance (cached)	30 ms	100 ms	300 ms
Place bet (cart — confirm)	200 ms	800 ms	2 s
List draws / results	50 ms	200 ms	500 ms
Admin approval actions	150 ms	500 ms	1.5 s
Report generation (large)	1 s	3 s	5 s

17.2 Load Test Plan (k6)

```
// load-test/draw-close.js
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '2m', target: 100 }, // ramp to 100
    { duration: '5m', target: 500 }, // normal peak
    { duration: '2m', target: 1200 }, // stress to 1200
    { duration: '5m', target: 1200 }, // sustain
    { duration: '3m', target: 0 }, // ramp down
  ],
  thresholds: {
    http_req_duration: ['p(95)<800', 'p(99)<2000'],
    http_req_failed: ['rate<0.01'], // < 1% errors
  },
};

export default function () {
  const token = login();
  const headers = { Authorization: `Bearer ${token}` };
  check(http.get(`${API}/wallet/balance`, { headers }), {
    'status 200': (r) => r.status === 200,
    'under 500ms': (r) => r.timings.duration < 500,
  });
  sleep(Math.random() * 2);
}
```

17.3 Pass Criteria

- p95 < 800ms across all endpoints at 1,200 concurrent users.
- Error rate < 1% sustained for 10 minutes at peak.
- Zero data inconsistency on wallet after test (automated reconciliation check).
- CPU usage across replicas stays below 70% at peak.

- Database connections never exceed 50 (PgBouncer limit).

18. Risk Mitigation

Risk	Likelihood	Impact	Mitigation
Scope creep during design phase	High	Medium	Lock Phase 1 scope with client sign-off. All new requests follow SOW Section 9.1 change process.
OTP gateway (zplusone) downtime	Medium	High	Implement fallback to SMS gateway. Cache last-used OTPs in Redis for verification window.
Peak traffic exceeds 1,200 users	Medium	High	Architecture scales to 5,000 by adding replicas. Pre-provision for 2× expected peak.
Wallet balance inconsistency	Low	Critical	DB constraints + SELECT FOR UPDATE + daily reconciliation job + audit log review.
iOS App Store review rejection	Medium	Medium	Follow guidelines strictly. Submit early (week 6) to allow 1–2 week review buffer.
Database runs out of disk	Low	High	Prometheus alert at 80%. Auto-scale volume in cloud. Archive old audit logs quarterly.
Celery task stuck / infinite loop	Low	Medium	Task time limit = 5 min. Alert on queue backlog. Idempotent settlement tasks safe to retry.
Developer illness / unavailability	Medium	Medium	Pair-programming on critical paths. Daily documentation. Code reviews ensure knowledge sharing.
Phase 1 design approval delayed	Medium	High	Start backend infrastructure in parallel during Phase 1. Design changes don't block infra work.
Payment gateway not provided by client	Medium	Low	Manual UPI + transaction ID approval model already in scope. Payment gateway is out-of-scope per SOW.

19. Implementation Roadmap — 10 Week Sprint Plan

From advance payment (19 April 2026) to go-live (30 June 2026) = 10 weeks. Two-week sprints, 5 sprints total.

Sprint 1 — 20 Apr to 3 May: Foundation

- **Designer:** UI/UX: User flows, wireframes for all user screens, design system.
- **Backend 1:** Docker compose setup, Postgres schema & migrations, Alembic, FastAPI skeleton, JWT auth scaffolding.
- **Backend 2:** Nginx config, PgBouncer, Redis setup, Prometheus + Grafana, CI pipeline.
- **Mobile:** Flutter project init, go_router, theme, dio client, secure storage, DI with get_it.

Sprint 2 — 4 May to 17 May: Core Auth & Wallet

- **Designer:** Admin panel wireframes, hi-fi designs for user app (login, dashboard, wallet, lottery).
- **Backend 1:** Auth use cases (register, login, reset password), OTP gateway integration, user service.
- **Backend 2:** Wallet service, transaction ledger, recharge with full validation, withdrawal request.
- **Mobile:** Auth feature (login, signup, OTP), wallet feature (balance, history). Integrate BLoC.

Sprint 3 — 18 May to 31 May: Lottery & Draws

- **Designer:** Hi-fi designs for admin panel — users, recharge queue, withdrawal queue, results.
- **Backend 1:** Draws, bets, cart, purchase confirm. Celery Beat for draw auto-close. Result entry workflow.
- **Backend 2:** Settlement task (credit winners), reports API (sales, cost, P&L), announcements.
- **Mobile:** Lottery feature (Dear-3, Kerala-4, Jackpot), cart, results, notifications.

Sprint 4 — 1 Jun to 14 Jun: Admin & Integration

- **Designer:** Design revisions from client feedback. Admin app polish. Handoff assets finalized.
- **Backend 1:** Admin endpoints complete: recharge approval, withdrawal approval, user mgmt, block, reset wallet, signup approval.
- **Backend 2:** Holiday management, audit log viewer, PDF report generation, rate limiting, security hardening.
- **Mobile:** Admin Flutter app — all screens. iOS build + TestFlight submission. Bug fixes.

Sprint 5 — 15 Jun to 30 Jun: Stabilize & Launch

- **Designer:** Final UI polish, asset export, documentation handoff.
- **Backend 1+2:** Load testing with k6, performance tuning, security review, production deployment.
- **Mobile:** Final bug fixes, Android Play Store submission, iOS App Store submission (or TestFlight live), E2E tests.
- **Team:** Production cutover on 30 June. Admin onboarding walkthrough with client. Go-Live.

Milestone Summary

Milestone	Target Date	Deliverable
M1 — Foundation Complete	3 May 2026	CI/CD running, DB schema done, skeleton apps booting
M2 — Design Approved	17 May 2026	All hi-fi designs signed off by client
M3 — Feature Complete	14 Jun 2026	All features in develop branch, staging deployed
M4 — Go-Live	30 Jun 2026	Production live, apps in stores, admin trained
M5 — Stabilization End	31 Aug 2026	All Phase-4 bug fixes closed, final source handoff

Sk6.0 System Design / Version 1.0 / April 2026

This document is confidential. Engineering use only.